Distributed Cache

Distributed Cache 與單機 Cache 的本質差異

單機 cache 的核心問題通常是 eviction 策略、TTL 設定、cache-aside 還是 write-through，以及如何避免 cache stampede。這些問題集中在記憶體管理與讀寫流程。

但一旦 cache 變成多節點 cluster，問題的重心會轉移。系統設計者必須開始處理節點分配、節點故障、網路延遲與流量分布不均等問題。此時 cache 已經不是單純的加速層，而是一個獨立的 distributed system。

舉例來說，一個電商平台原本只有一台 Redis，所有商品頁面資料都存在這台機器上。當流量成長到單機記憶體撐不住時，團隊決定升級成 Redis Cluster。從這一刻開始，問題不再只是「記憶體夠不夠」，而是「key 怎麼分配」、「某台機器掛掉時會不會影響整體」、「rebalancing 期間流量會不會暴增」。

可以用一句話概括這個階段的轉變：

Distributed cache 的難點不在於快，而在於它自己變成一個需要被設計與維運的分散式系統。

buildnoat.org

Sharding 與 Consistent Hashing

當 cache 只有一台機器時，所有 key 都存在同一個記憶體空間。升級成 cluster 之後，第一個必須解決的問題是 key 的分布。

常見做法是使用 consistent hashing。它的價值通常在擴容或縮容時才真正顯現。如果使用簡單的 hash(key) % N，當節點數量改變時，幾乎所有 key 都會重新映射，導致大量 cache miss。對於高流量系統來說，這等同於瞬間 cold start。

例如一個社交平台把使用者 profile 存在 distributed cache 裡。當原本 5 台 cache 節點擴容成 6 台時，如果沒有 consistent hashing，幾乎所有 profile cache 都會失效。接下來幾分鐘內，資料庫會承受平常數倍的流量。

Consistent hashing 的好處在於節點變動時只影響部分 key，降低 rebalancing 帶來的衝擊。不過，它只解決平均分布問題，並不解決流量偏斜。

Hot Key 問題

在實務上，distributed cache 的常見問題不是平均負載，而是極端負載。

例如一個熱門直播間、某位明星帳號、或首頁推薦資料，可能佔據整體流量的 20% 以上。即使 consistent hashing 把 key 均勻分布在節點上，只要這個 key 對應到某一台機器，那台機器就會成為 bottleneck。

常見解法包括：

- 對該 key 做多副本存放，讓 client 隨機讀取不同副本
- 在 application server 上加一層 local in-process cache
- 將單一 key 拆成多個邏輯 key（例如 user:123:profile:0/1/2）

這類問題通常發生在流量真的變大之後，而不是設計階段就能完全預測。因此 distributed cache 的設計不只要能擴展（scale），也要能應對分布不均（skew）。

Cache Replication 與 High Availability

很多團隊在早期會假設 cache 掛掉沒關係，反正可以從資料庫重建。但在高流量系統中，這種假設往往會造成事故。

當 cache 成為資料庫前面的保護層，一旦 cache cluster 全部失效，所有請求會直接回源到資料庫。如果原本 90% 的請求都由 cache 處理，資料庫可能會瞬間承受 10 倍流量。

例如一個訂票系統平時每秒 50k QPS，其中 45k 由 cache 命中。如果 cache cluster 因為配置錯誤全部重啟，剩下的 5k DB capacity 很快就會被壓垮。

因此 distributed cache 通常也需要 replication。例如每個 shard 有一個 replica，當 primary node 掛掉時，client 可以快速 failover。這裡的 replication 多半是 eventual consistency，因為 cache 本身並不承擔最終資料來源的責任。

不過 replication 也帶來新的問題，例如 failover 期間的短暫不一致、或 replica lag。設計時必須知道 cache 的一致性要求有多高。

Distributed Cache 的 Failure Modes

Distributed cache 的故障模式往往比資料庫更隱晦，因為它的失效通常是性能退化，而不是明顯 crash。

Cache Stampede

當大量 key 同時過期時，請求會同時 miss，進而打向資料庫。在 distributed 環境中，這個現象會被放大。多個 application server 同時 miss，多個 cache node 同時回源，資料庫壓力瞬間飆升。

buildmoat.org

常見做法是在 TTL 上加隨機 jitter，讓 key 不會同時過期，或在 application 層做 single-flight 機制，確保同一時間只有一個請求去回源。

Cold Start

當新節點加入 cluster 或整個 cluster 重啟時，cache 為空。這時候大量請求都會 miss。這種情況在夜間部署或自動擴容時很常見。

一些團隊會在新節點加入後先做預熱，或者限制流量逐步切換，避免瞬間把所有請求導到新節點。

Partial Node Failure

比起直接 crash，更常見的是某台 cache node 變慢但沒有完全失效。這會導致 client timeout，然後 retry。重試流量可能會打到其他節點或資料庫，造成連鎖效應。

因此 distributed cache 的 client 端通常需要合理的 timeout、重試上限，以及 circuit breaker，避免單點性能退化演變成整體崩潰。

Distributed Cache 一致性問題

Cache 天生就是一種放寬一致性的設計。但在 distributed 環境中，一致性管理更為複雜。

Cache Invalidation

單機 cache 已經很難處理 invalidation，多節點下更困難。例如寫入資料庫後發送 invalidation 訊息到多個 cache node，如果其中一個節點沒有收到這個訊息，就會持續保留舊資料。

很多系統最後採取的策略是 TTL 為主，invalidation 為輔。也就是即使 invalidation 失敗，資料最多 stale 一段可接受時間，而不是永久錯誤。

Write Path 與 Cache 同步

在 cache-aside 模式中，寫入資料庫後刪除 cache。如果刪除操作失敗，舊資料可能繼續存在於某些節點。若使用 write-through，則需要確保寫 cache 與寫資料庫的一致性，這在多節點下更難保證。

實務上，團隊通常會選擇簡單且可推理的快取模式，而不是追求強一致性的。

Multi-Region Distributed Cache

buildmost.org

當系統跨多個 region 時，通常每個 region 會有自己的 cache cluster，而不會跨 region 複製 cache。原因很實際：cache 是可重建的資料，不值得承擔跨區 replication 的複雜度與延遲。

例如一個全球服務的系統，美國與歐洲各有一套 cache。若歐洲 region 發生故障，流量切到美國，會出現短暫 cold cache，但資料仍然存在於資料庫中。

這種設計強調區域隔離，而不是全球一致的 cache。

在系統設計面試中如何談 Distributed Cache

在面試中，談 distributed cache 不應該只是說「用 Redis cluster」。比較完整的回答會包含：

- 為什麼需要分散 cache，而不是單機
- key 如何分片
- 節點擴容或縮容如何影響 cache 命中率
- cache cluster 故障時如何保護資料庫
- stampede 與 hot key 的處理策略
- 是否需要 replication，以及一致性要求

例如可以這樣描述：

我們會使用 sharded distributed cache，透過 consistent hashing 分配 key。每個 shard 設定 replica，避免單點失效。為了防止 stampede，TTL 加入隨機 jitter，並在應用層加入 single-flight 機制。對於高流量 hot key，會在應用層加入本地快取或做 key-level replication。如果整個 cache cluster 發生問題，系統會限制回源流量並啟動降級模式，保護資料庫。

這樣的回答顯示你理解 cache 是 performance optimization，但 distributed cache 本身需要完整的分散式設計思維。

bandmoat.org